

Introdução

Algoritmos, Dados e Tipos

Abstratos de Dados

Estruturas de Dados e Algoritmos

Prof. Sérgio Gorender

- Plano de Ensino Aprendizagem:

- Referências:

- N Ziviani. Projeto de Algoritmos com Implementações em Pascal e C. Cengage Learning. 3ª. Edição. 2012.
 - T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Steing. Algoritmos – Teoria e Prática. Ed. Capus. 3ª. Ed. 2012.
 - J. L. Szwarcfiter, L. Markenzon. Estruturas de Dados e Seus Algoritmos. Ed. LTC. 2ª. Ed. 2004.

- Avaliação:

- 3 notas com pesos iguais – 3 provas.
 - Em cada unidade poderão ser passados trabalhos também valendo para a nota.

- Inscrição no AVA

- Chave – mata40t01251

Introdução

- Algoritmo –
 - Descrição de um padrão de comportamento, expresso em termos de um conjunto finito de ações. (Dijkstra 1971)
- Solução de um problema:
 - Necessário escolher uma abstração da realidade, em geral definindo um conjunto de dados que representa a situação real.
 - Em seguida deve ser escolhida a forma de representar os dados, as quais são determinadas pelas operações a serem realizadas sobre os dados
- Programas são formulações concretas de algoritmos abstratos, baseados em representações e estruturas específicas de dados (Wirth 1976).

Uma linguagem de programação é uma técnica de notação para programar, com a intenção de servir de veículo tanto para a expressão do raciocínio algorítmico quanto para a execução automática de um algoritmo por um computador.

- Linguagem algorítmica baseada em pseudocódigo.
- Dados – representação da informação. Em geral definidos em termos de tipos e operações.
- Dados são armazenados em variáveis.

- Uma variável de uma linguagem de programação é uma abstração, composta de atributos como: nome, endereço de memória, valor, duração, escopo, tipo e tamanho, cada atributo com um valor associado.
- Variáveis em um programa são representadas em uma tabela de variáveis.
- O tipo define o conjunto de valores que pode ser atribuído à variável. O tipo de uma variável pode ser associado de forma estática (em tempo de compilação – **C**) ou de forma dinâmica (em tempo de execução – **Python**).
- Ao declarar o tipo de uma variável, definimos que valores podem ser armazenados, e que operações podem ser executadas sobre estes valores.

- Um tipo abstrato de dados pode ser visto como um modelo matemático, acompanhado das operações definidos sobre o modelo.
 - Ex. O conjunto dos inteiros acompanhado de suas operações.
 - A implementação de um algoritmo em uma linguagem de programação exige que se encontre alguma forma de representar o tipo abstrato de dados, em termos dos tipos de dados e dos operadores suportados pela linguagem.
 - A representação do modelo matemático por trás do tipo abstrato de dados é realizada através de uma estrutura de dados.
 - Um tipo abstrato de dados é uma forma de encapsular tipos de dados.

Tipos de dados

- Valores e suas operações:
 - Inteiros – representado como bits;
 - Operações: soma; subtração; multiplicação e divisão inteira.
 - int numero
 - Real – representado em ponto flutuante;
 - 2 inteiros em binário – mantissa e expoente, sendo:
 - mantissa o número e;
 - expoente a posição da vírgula.
 - Operações: soma, subtração, multiplicação, divisão, exponenciação, raiz.
 - real numero
 - Caractere – byte
 - Operações: soma (Concatenação)
 - char letra
 - Lógico – bit – binário – 0 ou 1;
 - boolean chave

- Tipos de dados estruturados:
 - Cadeia de caracteres – *string* – sequência de bytes.
 - Possui 1 byte para o tamanho e 1 byte para cada caractere.
 - Operações: posição, tamanho, concatenação e substring.
 - `char[20] nome`
 - Vetor – sequência de bytes.
 - Espaço de memória a partir de um endereço base.
 - Tamanho – depende do tipo de dado do vetor, e do tamanho (número de elementos) do vetor.
 - Operações – inserir valores, excluir valores, consultar valores.
 - `array(int)[30] lista`
 - Matriz – vetor de vetores.
 - `array(int)[20,20] matriz`

- Estruturas de controle:

- if <cond lógica> then
 <Comandos>
- if <cond lógica> then
 <Comandos>
 else
 <Comandos>
- do while <cond lógica>
 <Comandos>
- for <contagem>
 <Comandos>

- Atribuição - =

- Igual - ==

```
var = read()  
Print(var," ")
```

- Funções – subprogramas

- Forma de dividir um programa em sub-módulos, e reutilizar estes sub-módulos.
- Funções são chamadas para executar a partir do programa principal ou de sub-módulos.
- Uma função pode ter variáveis definidas internamente – variáveis locais.
- Variáveis definidas no programa principal podem ser acessadas de uma função – variáveis globais.
- Valores podem ser passados para uma função na chamada da função, como parâmetros:

function <nome>(<lista de parâmetros>)

 <comandos>

 return <parâmetros>

- Funções – subprogramas

- Passagem de parâmetros pode ser por valor ou por referência.
 - Passagem por valor – o valor armazenado na variável usada na chamada da função é transferido para a variável definida na lista de parâmetros.
 - Passagem por referência – a variável usada na chamada da função e a variável definida na lista de parâmetros compartilham a mesma referência na memória, sendo definidas nos mesmos endereços, e portanto, armazenam o mesmo valor, funcionando como nomes de variáveis diferentes para uma mesma variável na memória.
 - Existem linguagens que permitem as duas formas de passagem de parâmetros, e outros que só passam parâmetros por valor.
 - Vamos assumir a **passagem por valor**.
- Funções devolvem um ou mais valores no nome da função em sua chamada:
<lista de variáveis> = <nome> (<Lista de parâmetros>)